SOFTWARE ENGINEERING LAB

LAB 7

Fuzzing for Input Validation Bugs

Hotel Booking Application | Python + Hypothesis + Pytest

Student Name	Girish G N
SRN	PES1UG24CS571

Objective	Identify input-validation failures by generating varied inputs with Hypothesis, then update the application to handle invalid input safely.
Functions tested	`nights_stayed()`, `room_floor()`, and `booking_label()`.
Final result	**3 passed** — all three no-crash fuzz tests completed successfully.

This report documents the initial buggy behavior, the failures found by property-based testing, the corrected application run, and the final passing test run.

1. Environment Setup and Initial Bug

Hypothesis was installed and pytest was available in the Python 3.11 environment. The original application was run with an incomplete date input (12), causing the date-splitting logic to fail because it expected two values separated by a hyphen.

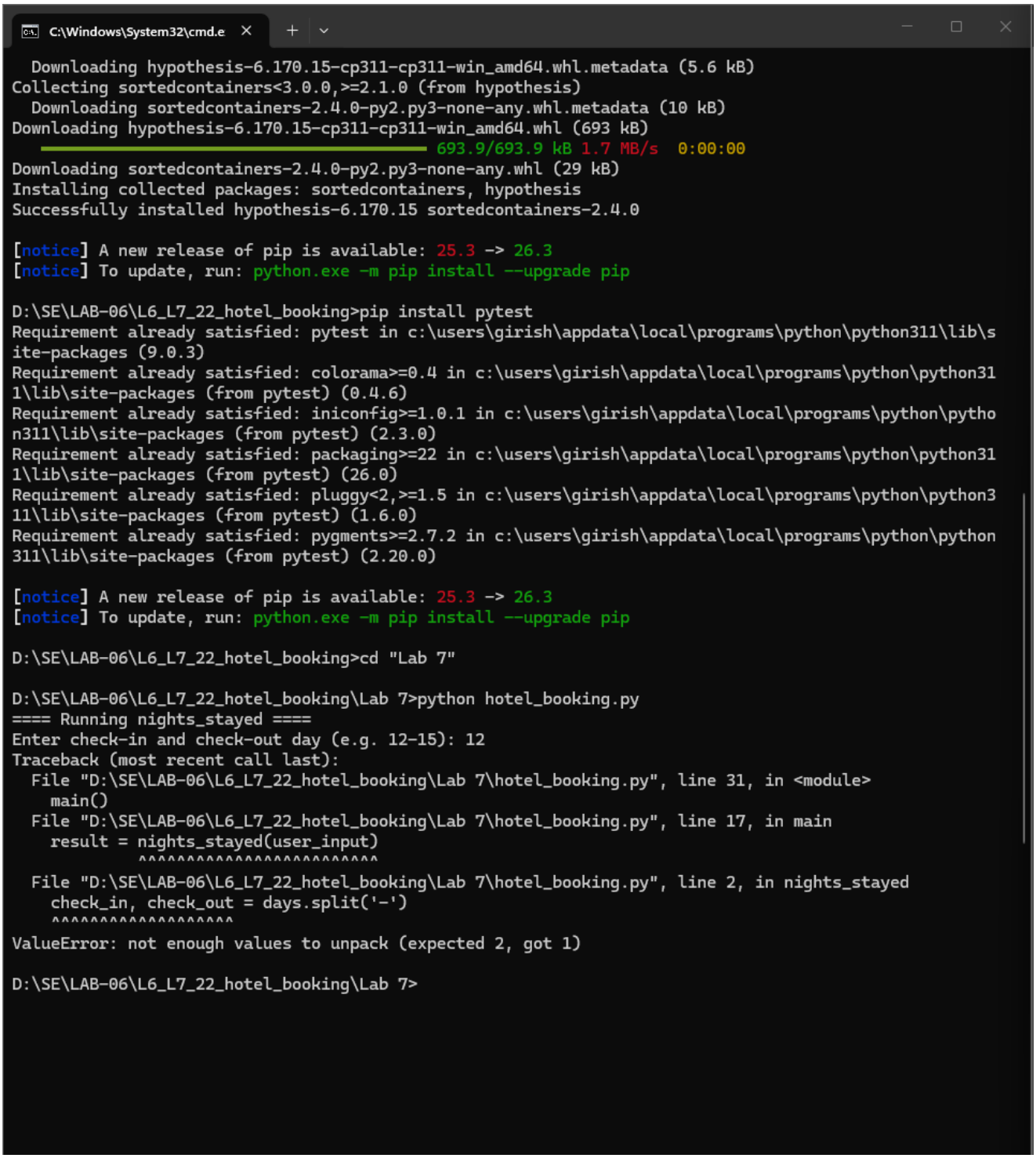


Figure 1. Environment setup and an example of the original application failing on malformed input.

2. Fuzzing the Original Application

Property-based tests were written for all three functions using Hypothesis text and None inputs. The initial run exposed crashes caused by malformed or missing values.

Function	Examples of discovered input problems
nights_stayed()	Missing separator, empty date component, and None input.
room_floor()	Missing room number, empty suffix, non-numeric floor character, and None input.
booking_label()	Empty guest list and None input.

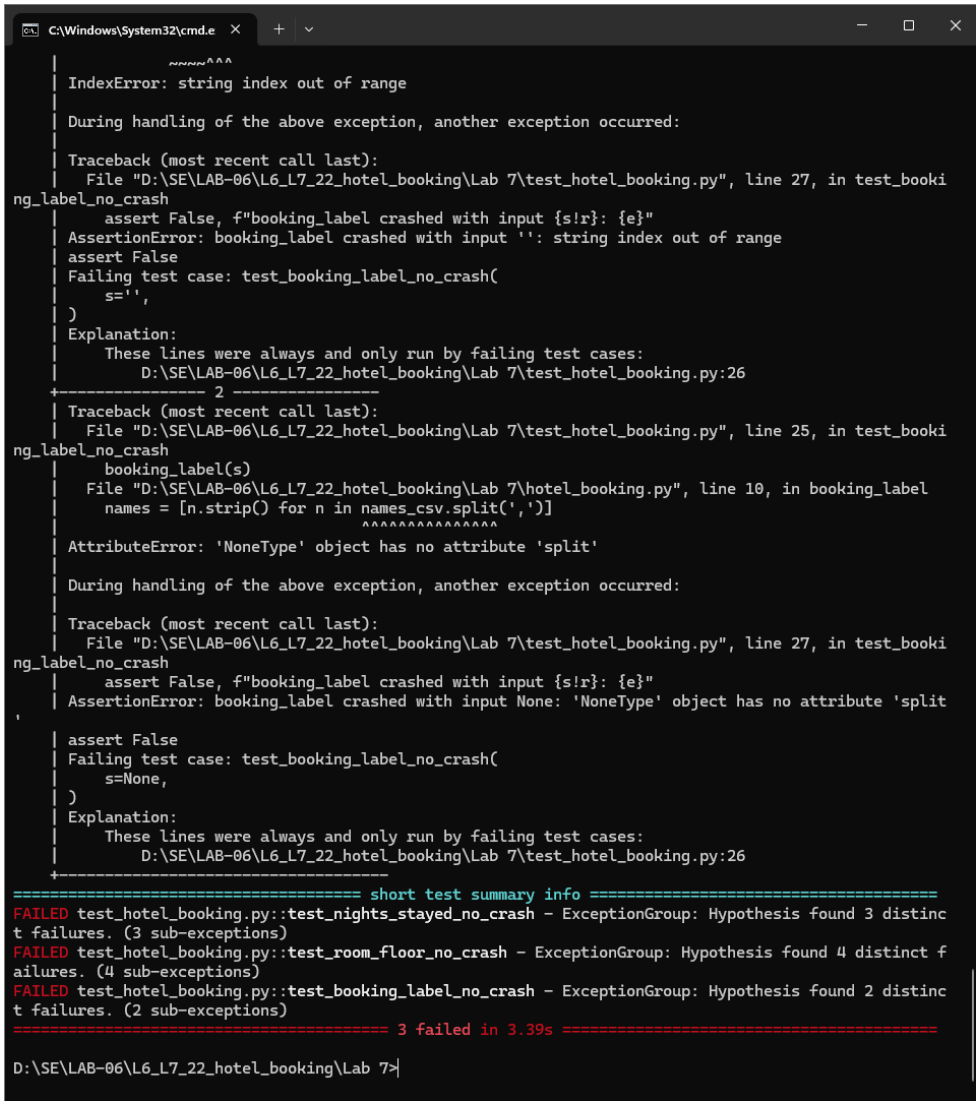


Figure 2. Initial Hypothesis run: all three tests failed, revealing input-handling defects.

3. Input Validation and Corrected Application

The application was updated to validate inputs before processing them and to return safe fallback values when inputs are malformed. The corrections include:

• **nights_stayed()** checks the input type, verifies the expected date format, checks for empty components, converts values safely, and rejects invalid day order.
• **room_floor()** checks the input type and room-code structure, then verifies that the room-number portion contains digits before converting it.
• **booking_label()** handles non-string and empty input, removes blank guest entries, and uses a fallback label when no valid names remain.

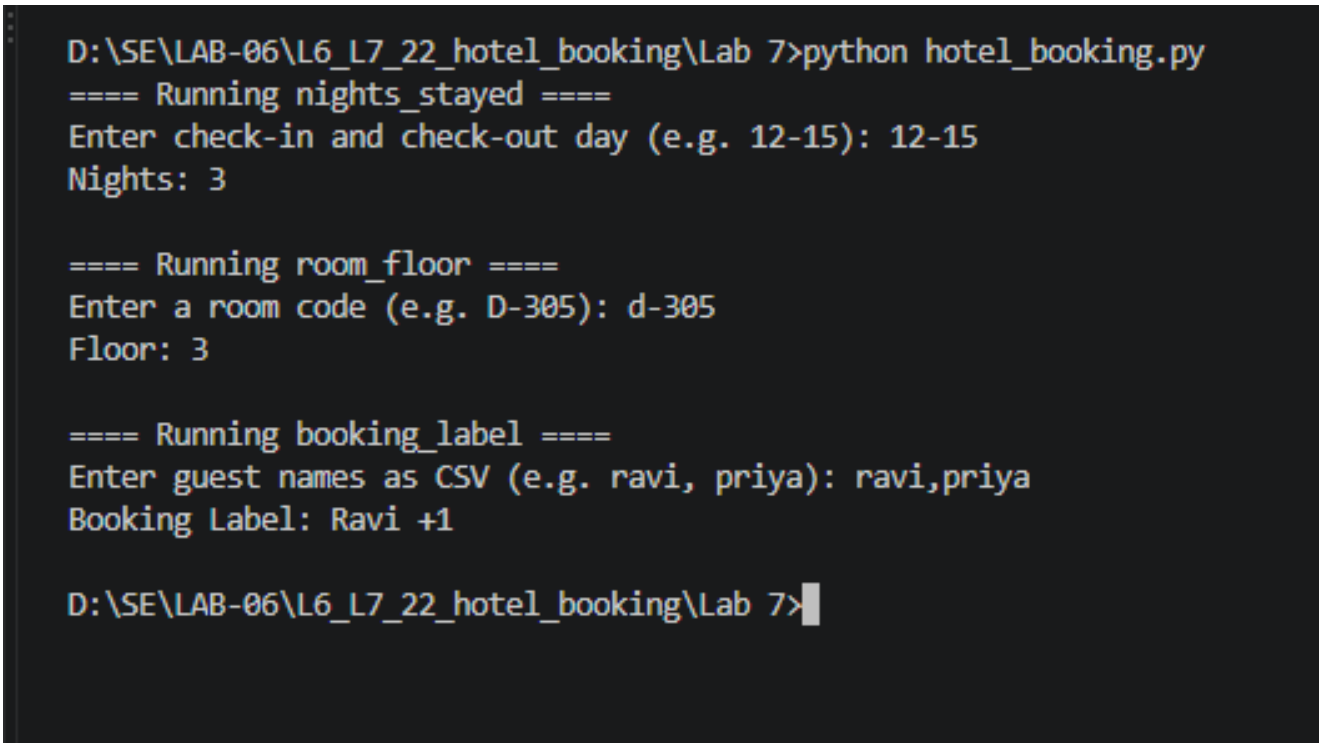


Figure 3. Corrected application run with sample inputs: 12-15, D-305, and ravi,priya.

Observed output: Nights = 3; Floor = 3; Booking Label = Ravi +1.

4. Final Fuzz Test Results

After updating the application, the same Hypothesis tests were executed again using pytest. All three tests passed, showing that the tested generated inputs no longer caused the functions to crash.

```
D:\SE\LAB-06\L6_L7_22_hotel_booking\Lab 7>python -m pytest test_hotel_booking.py -v

============================= test session starts =============================
platform win32 -- Python 3.11.6, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Girish\AppD
ata\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: D:\SE\LAB-06\L6_L7_22_hotel_booking\Lab 7
plugins: anyio-4.13.0, hypothesis-6.170.15, jaxtyping-0.3.10, asyncio-1.4.0, cov-7.
1.0, typeguard-4.5.2
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, as
yncio_default_test_loop_scope=function
collected 3 items

test_hotel_booking.py::test_nights_stayed_no_crash PASSED                [ 33%]
test_hotel_booking.py::test_room_floor_no_crash PASSED                   [ 66%]
test_hotel_booking.py::test_booking_label_no_crash PASSED                [100%]

============================== 3 passed in 1.22s ==============================

D:\SE\LAB-06\L6_L7_22_hotel_booking\Lab 7>
```

Figure 4. Final pytest output: all three fuzz tests passed.

FINAL RESULT	**3 passed** — nights_stayed, room_floor, and booking_label.

Conclusion

Hypothesis helped uncover input-validation bugs that were not obvious from normal example inputs. After adding defensive checks and safe fallback behavior, all three no-crash tests passed. These tests demonstrate crash resistance for generated inputs; they do not, by themselves, prove that every possible business rule or output is correct.